Digispark Keyboard Scripter Basic IDE

Input Prompting By: Mike Young – May 3, 2026
Using Gemini Pro 3.1
Last Update: August 2, 2026
YouTube: https://youtu.be/hyzvSi9E6LM
GitHub: https://github.com/myoung8223/dks

Introduction:

In this project we'll build a very basic, mostly portable (aside from a needing to install drivers), IDE (Integrated Development Environment) application for Windows that will allow writing new, open existing, saving, and "transpiling" keyboard scripts, similar to DuckyScript™, into Arduino ino files, compiling, and uploading those to a ATtiny85-based Digispark microcontroller, using arduino-cli. Such a device has usefulness in areas of automation and penetration testing.

Items needed:

- Windows PC
- Tiny C Compiler (only needed if building from source)
- Arduino CLI
- Digispark development board

See the **Windows Binary Release Version** section below for the binary release version for Windows. The Windows binary release can be downloaded from:

https://www.mikesshorts.com/misc/dks/dks.exe

Building from Source:

Start by creating a folder for this project. Call the folder "dks" for "Digispark Keyboard Scripter". Download the Tiny C Compiler and Arduino CLI zip files. Extract the Tiny C Compiler, "tcc", into its own folder within the "dks" folder and move the extracted "arduino-cli.exe" file to the "dks" folder. Within the "tcc" folder you should see "tcc.exe", the Tiny C Compiler main executable.

Create a text file in the "dks" folder called "arduino-cli.yaml", with the following contents…

arduino-cli.yaml

```
directories:
  data: ./portable_data
  downloads: ./portable_data/staging
  user: ./portable_data/user
board_manager:
  additional_urls:
    - https://raw.githubusercontent.com/ArminJo/DigistumpArduino/master/package_digistump_index.json
```

That file basically tells Arduino CLI to save everything in “portable_data” in the “dks” folder, as opposed to saving needed files in the user profile. This helps make this solution semi-portable, aside from a needed driver to install for the Digispark so that Windows can interface with it.

Next, the toolchain needs to be initialized. Open a command prompt and change directory to the “dks” folder for this project, then run the following commands…

cmd commands

```
arduino-cli core update-index --config-file arduino-cli.yaml
arduino-cli core download digistump:avr --config-file arduino-cli.yaml
```

The first command “updates the index” and the second command downloads the Digispark (Digistump) core. After the core is downloaded, you could take a copy of the dks folder for configuring portable installations, as at that point, nothing further needs to be downloaded from the Internet. Next, run this command to actually install the core and drivers (what “install.cmd” runs in the binary distribution version)…

cmd command

```
arduino-cli core install digistump:avr --config-file arduino-cli.yaml
```

Next, run this command to verify the installation…

cmd command

```
arduino-cli board listall digispark --config-file arduino-cli.yaml
```

You should see output like so…

cmd output

```
Board Name                               FQBN
Digispark (16mhz - No USB)               digistump:avr:digispark-tiny16
Digispark (1mhz - No USB)                digistump:avr:digispark-tiny1
Digispark (8mhz - No USB)                digistump:avr:digispark-tiny8
Digispark (Default - 16.5mhz)            digistump:avr:digispark-tiny
Digispark Pro (16 Mhz) (32 byte buffer) digistump:avr:digispark-pro32
Digispark Pro (16 Mhz) (64 byte buffer) digistump:avr:digispark-pro64
Digispark Pro (Default 16 Mhz)           digistump:avr:digispark-pro
```

If you see that, the portable compiler is configured properly and ready for use.

Optional Installation Confirmation:

Next, let’s create a “hello world” test to compile and upload to the Digispark. Create a folder in “dks” called “test_led”, and within that folder, create a text file called “test_led.ino”, containing the following text…

test_led.ino

```
void setup() { pinMode(1, OUTPUT); }
void loop() {
  digitalWrite(1, HIGH); delay(500);
  digitalWrite(1, LOW); delay(500);
}
```

Save that file and then run the following command…

cmd command

```
arduino-cli compile -b digistump:avr:digispark-tiny ./test_led --config-file arduino-cli.yaml
```

You should see output like so…

cmd output

```
Sketch uses 700 bytes (11%) of program storage space. Maximum is 6012 bytes.
Global variables use 9 bytes of dynamic memory.
```

Now, let's upload the compiled "hello world" test program to the Digispark. Make sure your Digispark board is not plugged into your computer, then run this command…

cmd command

```
arduino-cli upload -b digistump:avr:digispark-tiny ./test_led --config-file arduino-cli.yaml
```

You may get a Windows Security notification asking if you wish to allow public and private networks to access the "mdns-discovery.exe" application. You can click either "Allow" or "Cancel" to that notice. That application is a utility the Arduino toolchain uses to find boards over a network (like an ESP32 or a Wi-Fi-enabled Arduino). Since the Digispark is strictly USB, that functionality isn't needed. Further below in this guide there is an option to completely suppress that issue from the start by replacing "mdns-discovery.exe" with a dummy program.

Watch the command prompt. It should state…

cmd output

```
Running Digispark Uploader...
Plug in device now... (will timeout in 60 seconds)
```

Plug-in the Digispark board when you see that message. The program should then be uploaded and you should see output ending as follows…

cmd output

```
> Starting the user app ...
running: 100% complete
>> Micronucleus done. Thank you!
New upload port:  (default)
```

The board should then have a blinking red LED, in addition to its solid green power LED.

At this point, the dks.exe binary could be copied to the dks folder. However, if you wish to compile from the source code, proceed as follows.

Windows GUI IDE Program

Next, we need to write the Windows GUI C program, to be compiled using tcc, that will handle allowing the user to write, open, save, save as, and build and upload the result to the Digispark.

Create a file called “dks.c”, located in the “tcc” folder, with the following text content…

dks.c

```
// ============================================================================================================
// Digispark Keyboard Scripter IDE
// Copyright (c) 2026 Mike Young
// Licensed under the MIT License. See LICENSE file in the project root for full license information.
// ============================================================================================================

// ============================================================================
// Digispark Payload IDE & Transpiler
// ============================================================================

#include <windows.h>
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
#include <string.h>

// ============================================================================
// --- MANUAL DEFINITIONS (Bypassing missing commdlg.h & NLS for TCC) ---
// ============================================================================
typedef struct {
    DWORD        lStructSize;
    HWND         hwndOwner;
    HINSTANCE    hInstance;
    LPCSTR       lpstrFilter;
    LPSTR        lpstrCustomFilter;
    DWORD        nMaxCustFilter;
    DWORD        nFilterIndex;
    LPSTR        lpstrFile;
    DWORD        nMaxFile;
    LPSTR        lpstrFileTitle;
    DWORD        nMaxFileTitle;
    LPCSTR       lpstrInitialDir;
    LPCSTR       lpstrTitle;
    DWORD        Flags;
    WORD         nFileOffset;
    WORD         nFileExtension;
    LPCSTR       lpstrDefExt;
    LPARAM       lCustData;
    void*        lpfnHook;
    LPCSTR       lpTemplateName;
} OPENFILENAME;

// Dialog Flags for File Open/Save behavior
```

```
#define OFN_HIDEREADONLY     0x00000004
#define OFN_OVERWRITEPROMPT  0x00000002
#define OFN_PATHMUSTEXIST    0x00000800
#define OFN_FILEMUSTEXIST    0x00001000
#define OFN_EXPLORER         0x00080000

// SS_NOPREFIX makes STATIC controls render '&' literally instead of
// treating it as a keyboard-mnemonic prefix. Guarded in case TCC omits it.
#ifndef SS_NOPREFIX
#define SS_NOPREFIX 0x00000080
#endif

// Tell the compiler these functions exist in the comdlg32 library
BOOL WINAPI GetOpenFileNameA(OPENFILENAME *lpofn);
BOOL WINAPI GetSaveFileNameA(OPENFILENAME *lpofn);

#define GetOpenFileName GetOpenFileNameA
#define GetSaveFileName GetSaveFileNameA

// ShellExecuteA lives in shell32
HINSTANCE WINAPI ShellExecuteA(HWND, LPCSTR, LPCSTR, LPCSTR, LPCSTR, INT);
#ifndef ShellExecute
#define ShellExecute ShellExecuteA
#endif

// Unicode/ANSI String Conversion (missing in some TCC winapi header subsets)
#ifndef CP_ACP
#define CP_ACP 0
#endif
int WINAPI WideCharToMultiByte(UINT, DWORD, LPCWSTR, int, LPSTR, int, LPCSTR, LPBOOL);

// CheckMenuRadioItem may be absent from minimal TCC winapi headers
BOOL WINAPI CheckMenuRadioItem(HMENU, UINT, UINT, UINT, UINT);

// ============================================================================
// --- APPLICATION CONSTANTS & GLOBALS ---
// ============================================================================

// Menu Event IDs
#define IDM_NEW 1001
#define IDM_OPEN 1002
#define IDM_SAVE 1003
#define IDM_FLASH 1004
#define IDM_TOGGLE_DARK 1005
#define IDM_TOGGLE_WRAP 1006
#define IDM_SAVE_AS 1007
#define IDM_ABOUT 1008

// Typing-delay preset menu IDs (safePrint inter-key delay)
#define IDM_DELAY_0  1010
#define IDM_DELAY_1  1011
#define IDM_DELAY_5  1012
#define IDM_DELAY_10 1013
#define IDM_DELAY_15 1014

// About-dialog control IDs
#define IDC_ABOUT_VISIT 2001
#define IDC_ABOUT_CLOSE 2002
```

```
// Application version string (shown in the About box; bump this on each release)
#define APP_VERSION "18"

// GitHub repository URL (shown in the About box and opened by the browser prompt)
#define GITHUB_URL "https://github.com/myoung8223/dks"

// Practical flash ceiling (bytes). The Micronucleus bootloader actually on the
// device can leave LESS usable flash than arduino-cli / the core reports as its
// "Maximum", so a sketch that "fits" per the compiler can still fail to upload
// or run. Sketches larger than this are blocked before upload. Adjust to match
// the bootloader on your hardware.
#define SAFE_FLASH_MAX 6012

// USB HID Modifier Bitmasks (Used by DigiKeyboard)
#define MOD_CONTROL_LEFT  (1 << 0)
#define MOD_SHIFT_LEFT    (1 << 1)
#define MOD_ALT_LEFT      (1 << 2)
#define MOD_GUI_LEFT      (1 << 3)

// Global Handles
HWND hEdit;                         // The main text editor window
char currentFile[MAX_PATH] = "";    // Tracks the currently opened file path
char iniPath[MAX_PATH] = "";        // Tracks the path to the settings.ini file
HBRUSH hEditBkBrush = NULL;         // Brush for Dark Mode background
WNDPROC oldEditProc = NULL;         // Subclass window procedure pointer for the editor
HMENU gDelayMenu = NULL;            // Handle to the "Typing Delay" submenu (for radio checks)

int isDarkMode = 0;                 // Dark mode toggle state
int isWordWrap = 1;                 // Word wrap toggle state
int safePrintDelay = 15;            // Inter-key delay (ms) injected into safePrint; 0 = none

// ============================================================================
// --- CONFIGURATION HELPERS ---
// ============================================================================

// Builds the absolute path to settings.ini in the same folder as the executable
void InitIniPath() {
    GetModuleFileName(NULL, iniPath, MAX_PATH);
    char *lastSlash = strrchr(iniPath, '\\');
    if (lastSlash) {
        strcpy(lastSlash + 1, "settings.ini");
    }
}

// Reads settings from the INI file
void LoadSettings() {
    isDarkMode = GetPrivateProfileInt("Settings", "DarkMode", 0, iniPath);
    isWordWrap = GetPrivateProfileInt("Settings", "WordWrap", 1, iniPath);
    safePrintDelay = GetPrivateProfileInt("Settings", "SafePrintDelay", 15, iniPath);
}

// Writes settings to the INI file
void SaveSettings() {
    char buffer[16];
    sprintf(buffer, "%d", isDarkMode);
    WritePrivateProfileString("Settings", "DarkMode", buffer, iniPath);
    sprintf(buffer, "%d", isWordWrap);
```

```
    WritePrivateProfileString("Settings", "WordWrap", buffer, iniPath);
    sprintf(buffer, "%d", safePrintDelay);
    WritePrivateProfileString("Settings", "SafePrintDelay", buffer, iniPath);
}

// ============================================================================
// --- STRING UTILITIES ---
// ============================================================================

// Helper: Aggressive Trim
// Removes leading and trailing whitespace/newlines from a string.
char* clean_str(char *str) {
    char *end;
    // Trim leading space
    while(isspace((unsigned char)*str)) str++;
    if(*str == 0) return str; // All spaces?

    // Trim trailing space
    end = str + strlen(str) - 1;
    while(end > str && isspace((unsigned char)*end)) end--;
    end[1] = '\0';
    return str;
}

// Helper: Writes a C-string-escaped copy of src to the output file.
// Ensures backslashes, quotes, tabs, and newlines in a payload survive
// intact when embedded inside a "..." literal in the generated sketch.
void fputs_escaped(const char *src, FILE *out) {
    for (const char *p = src; *p != '\0'; p++) {
        switch (*p) {
            case '\\': fputs("\\\\", out); break;
            case '\"': fputs("\\\"", out); break;
            case '\n': fputs("\\n", out);  break;
            case '\r': fputs("\\r", out);  break;
            case '\t': fputs("\\t", out);  break;
            default:   fputc(*p, out);     break;
        }
    }
}

// ============================================================================
// --- TRANSPILER ENGINE ---
// ============================================================================

int transpile(const char* inputFilename) {
    int current_mod = 0;
    int default_delay = 0;

    FILE *in = fopen(inputFilename, "r");
    CreateDirectory("sketch", NULL);
    FILE *out = fopen("sketch\\sketch.ino", "w");

    if (!in)  { if (out) fclose(out); return 0; }
    if (!out) { fclose(in); return 0; }

    fprintf(out, "#include \"DigiKeyboard.h\"\n");
    fprintf(out, "#include <avr/pgmspace.h>\n\n");
    fprintf(out, "// --- USB HID Constants (defined only if the core omits them) ---\n");
```

```
    fprintf(out, "#ifndef KEY_ENTER\n#define KEY_ENTER 40\n#endif\n");
    fprintf(out, "#ifndef KEY_ESC\n#define KEY_ESC 41\n#endif\n");
    fprintf(out, "#ifndef KEY_ESCAPE\n#define KEY_ESCAPE 41\n#endif\n");
    fprintf(out, "#ifndef KEY_BACKSPACE\n#define KEY_BACKSPACE 42\n#endif\n");
    fprintf(out, "#ifndef KEY_TAB\n#define KEY_TAB 43\n#endif\n");
    fprintf(out, "#ifndef KEY_SPACE\n#define KEY_SPACE 44\n#endif\n");
    fprintf(out, "#ifndef KEY_MINUS\n#define KEY_MINUS 45\n#endif\n");
    fprintf(out, "#ifndef KEY_EQUAL\n#define KEY_EQUAL 46\n#endif\n");
    fprintf(out, "#ifndef KEY_DELETE\n#define KEY_DELETE 76\n#endif\n");
    fprintf(out, "#ifndef KEY_RIGHT\n#define KEY_RIGHT 79\n#endif\n");
    fprintf(out, "#ifndef KEY_LEFT\n#define KEY_LEFT 80\n#endif\n");
    fprintf(out, "#ifndef KEY_DOWN\n#define KEY_DOWN 81\n#endif\n");
    fprintf(out, "#ifndef KEY_UP\n#define KEY_UP 82\n#endif\n\n");

    fprintf(out, "// Helper: Rate-limits typing to prevent dropped keystrokes (PROGMEM optimized)\n");
    fprintf(out, "void safePrint(const char* p) {\n");
    fprintf(out, "  while (true) {\n");
    fprintf(out, "    unsigned char c = pgm_read_byte(p++);\n");
    fprintf(out, "    if (c == 0) break;\n");
    fprintf(out, "    DigiKeyboard.print((char)c);\n");
    if (safePrintDelay > 0)
        fprintf(out, "    DigiKeyboard.delay(%d);\n", safePrintDelay);
    fprintf(out, "  }\n");
    fprintf(out, "}\n\n");

    fprintf(out, "// Helper: Blinks N times, then pauses 2s, repeating until Pin 2 goes LOW\n");
    fprintf(out, "void waitForButtonBlink(int count) {\n");
    fprintf(out, "  while(digitalRead(2) == HIGH) {\n");
    fprintf(out, "    for(int i=0; i < count; i++) {\n");
    fprintf(out, "      digitalWrite(1, HIGH); DigiKeyboard.delay(200);\n");
    fprintf(out, "      digitalWrite(1, LOW);  DigiKeyboard.delay(200);\n");
    fprintf(out, "      if(digitalRead(2) == LOW) return;\n");
    fprintf(out, "    }\n");
    fprintf(out, "    for(int j=0; j < 40; j++) {\n");
    fprintf(out, "      DigiKeyboard.delay(50); DigiKeyboard.update();\n");
    fprintf(out, "      if(digitalRead(2) == LOW) return;\n");
    fprintf(out, "    }\n");
    fprintf(out, "  }\n");
    fprintf(out, "  DigiKeyboard.delay(200);\n");
    fprintf(out, "}\n\n");

    fprintf(out, "void setup() {\n");
    fprintf(out, "  DigiKeyboard.sendKeyStroke(0);\n");
    fprintf(out, "  DigiKeyboard.delay(500);\n");
    fprintf(out, "  pinMode(1, OUTPUT);\n");
    fprintf(out, "  pinMode(2, INPUT_PULLUP);\n");
    fprintf(out, "}\n\nvoid loop() {\n");
    fprintf(out, "  DigiKeyboard.update();\n  DigiKeyboard.delay(3000);\n\n");

    char line[256];
    while (fgets(line, sizeof(line), in)) {
        char temp[256];
        strcpy(temp, line);

        int in_quotes = 0;
        for (int i = 0; temp[i] != '\0'; i++) {
            if (temp[i] == '\"') in_quotes = !in_quotes;
            if (temp[i] == '#' && !in_quotes) {
```

```
            temp[i] = '\0';
            break;
        }
    }

    char *trimmed = clean_str(temp);
    if (trimmed[0] == '\0') continue;

    char cmd[256];
    strcpy(cmd, trimmed);
    _strupr(cmd);

    int is_keyboard_action = 0;

    if (strncmp(cmd, "DEFAULTDELAY", 12) == 0) {
        sscanf(cmd, "DEFAULTDELAY %d", &default_delay);
        continue;
    }

    if (strncmp(cmd, "STRING(", 7) == 0) {
        char *start = strchr(trimmed, '\"');
        char *end = strrchr(trimmed, '\"');

        if (start && end && start != end) {
            *end = '\0';
            fputs("  safePrint(PSTR(\"", out);
            fputs_escaped(start + 1, out);
            fputs("\"));\n", out);
            is_keyboard_action = 1;
            *end = '\"';
        } else {
            char errorMsg[512];
            sprintf(errorMsg, "Syntax error (missing quotes):\n\n%s", trimmed);
            MessageBox(NULL, errorMsg, "Transpiler Crash", MB_ICONERROR);
            fclose(in); fclose(out); return 0;
        }
    }
    else if (strncmp(cmd, "KEYDOWN(", 8) == 0) {
        char k1[32];
        if (sscanf(cmd, "KEYDOWN(%31[^)])", k1) >= 1) {
            char *ck1 = clean_str(k1);
            if (strcmp(ck1, "CONTROL") == 0)      current_mod |= MOD_CONTROL_LEFT;
            else if (strcmp(ck1, "ALT") == 0)     current_mod |= MOD_ALT_LEFT;
            else if (strcmp(ck1, "SHIFT") == 0)   current_mod |= MOD_SHIFT_LEFT;
            else if (strcmp(ck1, "GUI") == 0)     current_mod |= MOD_GUI_LEFT;
            else {
                fprintf(out, "  DigiKeyboard.sendKeyPress(KEY_%s, %d);\n", ck1, current_mod);
                is_keyboard_action = 0;
                goto trigger_delay;
            }
            fprintf(out, "  DigiKeyboard.sendKeyPress(0, %d);\n", current_mod);
            is_keyboard_action = 0;
        }
    }
    else if (strncmp(cmd, "KEYUP()", 7) == 0) {
        current_mod = 0;
        fprintf(out, "  DigiKeyboard.sendKeyPress(0, 0);\n");
        is_keyboard_action = 1;
```

```
        }
        else if (strncmp(cmd, "KEYS(", 5) == 0) {
            char inner[128] = {0};
            if (sscanf(cmd, "KEYS(%127[^)])", inner) == 1) {
                char *t1 = strtok(inner, ",");
                char *t2 = strtok(NULL, ",");
                char *t3 = strtok(NULL, ",");

                if (t1) t1 = clean_str(t1);
                if (t2) t2 = clean_str(t2);
                if (t3) t3 = clean_str(t3);

                if (t1 && t2 && t3) {
                    fprintf(out, "  DigiKeyboard.sendKeyPress(KEY_%s, MOD_%s_LEFT | MOD_%s_LEFT);\n", t3,
t1, t2);
                    fprintf(out, "  DigiKeyboard.delay(100);\n");
                    fprintf(out, "  DigiKeyboard.sendKeyPress(0, 0);\n");
                } else if (t1 && t2) {
                    fprintf(out, "  DigiKeyboard.sendKeyStroke(KEY_%s, MOD_%s_LEFT);\n", t2, t1);
                } else if (t1) {
                    fprintf(out, "  DigiKeyboard.sendKeyStroke(KEY_%s);\n", t1);
                }
                is_keyboard_action = 1;
            }
        }
        else if (strncmp(cmd, "KEY(", 4) == 0) {
            char k1[32];
            if (sscanf(cmd, "KEY(%31[^)])", k1) >= 1) {
                if (current_mod > 0) {
                    fprintf(out, "  DigiKeyboard.sendKeyPress(KEY_%s, %d);\n", clean_str(k1), current_mod);
                    fprintf(out, "  DigiKeyboard.delay(50);\n");
                    fprintf(out, "  DigiKeyboard.sendKeyPress(0, %d);\n", current_mod);
                } else {
                    fprintf(out, "  DigiKeyboard.sendKeyStroke(KEY_%s);\n", clean_str(k1));
                }
                is_keyboard_action = 1;
            }
        }
        else if (strncmp(cmd, "DELAY(", 6) == 0) {
            int ms;
            if (sscanf(cmd, "DELAY(%d)", &ms) >= 1) {
                fprintf(out, "  DigiKeyboard.delay(%d);\n", ms);
            }
        }
        else if (strncmp(cmd, "WAITBLINK(", 10) == 0) {
            int blinks = 0;
            if (sscanf(cmd, "WAITBLINK(%d)", &blinks) >= 1) {
                fprintf(out, "  waitForButtonBlink(%d);\n", blinks);
            }
        }
        else if (strcmp(cmd, "LED(ON)") == 0) {
            fprintf(out, "  digitalWrite(1, HIGH);\n");
        }
        else if (strcmp(cmd, "LED(OFF)") == 0) {
            fprintf(out, "  digitalWrite(1, LOW);\n");
        }
        else if (strcmp(cmd, "WAIT()") == 0) {
```

```
            fprintf(out, "  while(digitalRead(2) == HIGH) { DigiKeyboard.update(); DigiKeyboard.delay(50);
}\n");
            fprintf(out, "  DigiKeyboard.delay(200);\n");
        }
        else {
            char errorMsg[512];
            sprintf(errorMsg, "The transpiler encountered a syntax error and doesn't understand this
line:\n\n%s", trimmed);
            MessageBox(NULL, errorMsg, "Transpiler Error", MB_ICONERROR);
            fclose(in); fclose(out); return 0;
        }

        trigger_delay:
        if (is_keyboard_action && default_delay > 0) {
            fprintf(out, "  DigiKeyboard.delay(%d); // Auto-delay\n", default_delay);
        }
    }

    fprintf(out, "\n  for(;;){ DigiKeyboard.delay(1000); }\n}\n");
    fclose(in);
    fclose(out);
    return 1;
}

// ============================================================================
// --- FILE & PROCESS HELPERS ---
// ============================================================================

void SaveFile(HWND hwnd, const char* filename) {
    int len = GetWindowTextLength(hEdit);
    if (len > 0) {
        char* buffer = (char*)malloc(len + 1);
        GetWindowText(hEdit, buffer, len + 1);
        FILE* fp = fopen(filename, "wb");
        if (fp) {
            fwrite(buffer, 1, len, fp);
            fclose(fp);
        }
        free(buffer);
    } else {
        FILE* fp = fopen(filename, "wb");
        if (fp) fclose(fp);
    }
}

void LoadFile(HWND hwnd, const char* filename) {
    FILE* fp = fopen(filename, "rb");
    if (fp) {
        fseek(fp, 0, SEEK_END);
        long size = ftell(fp);
        rewind(fp);

        char* buffer = (char*)malloc(size + 1);
        size_t bytesRead = fread(buffer, 1, size, fp);
        buffer[bytesRead] = '\0';
        fclose(fp);

        char* normalized = (char*)malloc(bytesRead * 2 + 1);
```

```
        size_t j = 0;
        size_t i = 0;
        while (i < bytesRead) {
            // Normalize each line break to a single \r\n. Unlike a run-consuming
            // loop, this treats \r\n as ONE break, and lone \r or \n as one each,
            // so consecutive breaks (blank lines) are preserved rather than merged.
            if (buffer[i] == '\r') {
                normalized[j++] = '\r';
                normalized[j++] = '\n';
                i += (i + 1 < bytesRead && buffer[i + 1] == '\n') ? 2 : 1; // eat paired \n
            } else if (buffer[i] == '\n') {
                normalized[j++] = '\r';
                normalized[j++] = '\n';
                i += 1;
            } else {
                normalized[j++] = buffer[i++];
            }
        }
        normalized[j] = '\0';

        SetWindowText(hEdit, normalized);
        free(buffer);
        free(normalized);
    }
}

int RunCommandHidden(const char* cmd) {
    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESHOWWINDOW;
    si.wShowWindow = SW_HIDE;
    ZeroMemory(&pi, sizeof(pi));

    char cmdBuffer[1024];
    strncpy(cmdBuffer, cmd, sizeof(cmdBuffer) - 1);
    cmdBuffer[sizeof(cmdBuffer) - 1] = '\0';

    if (CreateProcess(NULL, cmdBuffer, NULL, NULL, FALSE, CREATE_NO_WINDOW, NULL, NULL, &si, &pi)) {
        WaitForSingleObject(pi.hProcess, INFINITE);
        DWORD exitCode;
        GetExitCodeProcess(pi.hProcess, &exitCode);
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
        return (int)exitCode;
    }
    return -1;
}

int extract_line(const char* src, const char* needle, char* dest, int destSize) {
    const char* p = strstr(src, needle);
    if (!p) { if (destSize > 0) dest[0] = '\0'; return 0; }
    int i = 0;
    while (p[i] != '\0' && p[i] != '\r' && p[i] != '\n' && i < destSize - 1) {
        dest[i] = p[i];
        i++;
    }
```

```
    dest[i] = '\0';
    return 1;
}

int RunCommandCapture(const char* cmd, char* outBuf, int outBufSize) {
    if (outBuf && outBufSize > 0) outBuf[0] = '\0';

    char tmpDir[MAX_PATH], tmpFile[MAX_PATH];
    GetTempPath(MAX_PATH, tmpDir);
    if (!GetTempFileName(tmpDir, "dig", 0, tmpFile)) return -1;

    SECURITY_ATTRIBUTES sa;
    ZeroMemory(&sa, sizeof(sa));
    sa.nLength = sizeof(sa);
    sa.bInheritHandle = TRUE;

    HANDLE hFile = CreateFile(tmpFile, GENERIC_WRITE, FILE_SHARE_READ | FILE_SHARE_WRITE,
                              &sa, CREATE_ALWAYS, FILE_ATTRIBUTE_TEMPORARY, NULL);
    if (hFile == INVALID_HANDLE_VALUE) { DeleteFile(tmpFile); return -1; }

    STARTUPINFO si;
    PROCESS_INFORMATION pi;
    ZeroMemory(&si, sizeof(si));
    si.cb = sizeof(si);
    si.dwFlags = STARTF_USESHOWWINDOW | STARTF_USESTDHANDLES;
    si.wShowWindow = SW_HIDE;
    si.hStdInput  = NULL;
    si.hStdOutput = hFile;
    si.hStdError  = hFile;
    ZeroMemory(&pi, sizeof(pi));

    char cmdBuffer[1024];
    strncpy(cmdBuffer, cmd, sizeof(cmdBuffer) - 1);
    cmdBuffer[sizeof(cmdBuffer) - 1] = '\0';

    int exitCode = -1;
    if (CreateProcess(NULL, cmdBuffer, NULL, NULL, TRUE, CREATE_NO_WINDOW, NULL, NULL, &si, &pi)) {
        WaitForSingleObject(pi.hProcess, INFINITE);
        DWORD dwExit;
        GetExitCodeProcess(pi.hProcess, &dwExit);
        exitCode = (int)dwExit;
        CloseHandle(pi.hProcess);
        CloseHandle(pi.hThread);
    }

    CloseHandle(hFile);

    if (outBuf && outBufSize > 0) {
        FILE* fp = fopen(tmpFile, "rb");
        if (fp) {
            fseek(fp, 0, SEEK_END);
            long fsize = ftell(fp);
            long want  = (long)outBufSize - 1;
            if (fsize > want) {
                fseek(fp, fsize - want, SEEK_SET);
            } else {
                rewind(fp);
            }
```

```
            size_t n = fread(outBuf, 1, (size_t)outBufSize - 1, fp);
            outBuf[n] = '\0';
            fclose(fp);
        }
    }

    DeleteFile(tmpFile);
    return exitCode;
}

// ============================================================================
// --- EDITOR SUBCLASS HOOK (Robust Clipboard Normalization) ---
// ============================================================================

// Helper to safely normalize and inject text into the ANSI Edit Control
void InsertNormalizedText(HWND hwnd, const char* text) {
    if (!text) return;
    size_t len = strlen(text);
    char* normalized = (char*)malloc(len * 2 + 1);
    size_t j = 0;

    for (size_t i = 0; i < len; i++) {
        // Handle \r (Mac), \n (Unix), and \r\n (Windows) universally
        if (text[i] == '\r') {
            normalized[j++] = '\r';
            normalized[j++] = '\n';
            if (text[i + 1] == '\n') i++; // Skip paired \n
        } else if (text[i] == '\n') {
            normalized[j++] = '\r';
            normalized[j++] = '\n';
        } else {
            normalized[j++] = text[i];
        }
    }
    normalized[j] = '\0';

    // Using SendMessageA ensures the ANSI control processes it correctly
    SendMessageA(hwnd, EM_REPLACESEL, TRUE, (LPARAM)normalized);
    free(normalized);
}

LRESULT CALLBACK EditSubProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    // ------------------------------------------------------------------------
    // Intercept Ctrl+A for "Select All"
    // ------------------------------------------------------------------------
    if (msg == WM_KEYDOWN) {
        if (wParam == 'A' && (GetKeyState(VK_CONTROL) & 0x8000)) {
            SendMessage(hwnd, EM_SETSEL, 0, -1);
            return 0;
        }
    }

    // ------------------------------------------------------------------------
    // Suppress the system "ding" sound on WM_CHAR for Ctrl+A
    // ------------------------------------------------------------------------
    if (msg == WM_CHAR) {
        if ((GetKeyState(VK_CONTROL) & 0x8000) && (wParam == 1 || wParam == 'a' || wParam == 'A')) {
            return 0;
```

```
        }
    }

    // ---------------------------------------------------------------------
    // Custom Paste Handling (Robust Clipboard Normalization)
    // ---------------------------------------------------------------------
    if (msg == WM_PASTE) {
        if (OpenClipboard(hwnd)) {
            // Priority 1: CF_UNICODETEXT (Notepad++, Web Browsers, VS Code)
            if (IsClipboardFormatAvailable(CF_UNICODETEXT)) {
                HANDLE hClip = GetClipboardData(CF_UNICODETEXT);
                if (hClip) {
                    WCHAR* clipText = (WCHAR*)GlobalLock(hClip);
                    if (clipText) {
                        int ansiLen = WideCharToMultiByte(CP_ACP, 0, clipText, -1, NULL, 0, NULL, NULL);
                        if (ansiLen > 0) {
                            char* ansiText = (char*)malloc(ansiLen);
                            WideCharToMultiByte(CP_ACP, 0, clipText, -1, ansiText, ansiLen, NULL, NULL);
                            InsertNormalizedText(hwnd, ansiText);
                            free(ansiText);
                        }
                        GlobalUnlock(hClip);
                    }
                }
            }
            // Priority 2: Standard CF_TEXT Fallback
            else if (IsClipboardFormatAvailable(CF_TEXT)) {
                HANDLE hClip = GetClipboardData(CF_TEXT);
                if (hClip) {
                    char* clipText = (char*)GlobalLock(hClip);
                    if (clipText) {
                        InsertNormalizedText(hwnd, clipText);
                        GlobalUnlock(hClip);
                    }
                }
            }
            CloseClipboard();
            return 0; // Handled paste message completely
        }
    }

    // Pass everything else to the original Edit control procedure
    return CallWindowProc(oldEditProc, hwnd, msg, wParam, lParam);
}

// Recreates the Edit control to toggle styles like Word Wrap
void RecreateEditor(HWND hwndParent) {
    int len = GetWindowTextLength(hEdit);
    char* buffer = NULL;
    if (len > 0) {
        buffer = (char*)malloc(len + 1);
        GetWindowText(hEdit, buffer, len + 1);
    }

    HFONT hCurrentFont = (HFONT)SendMessage(hEdit, WM_GETFONT, 0, 0);

    DWORD editStyle = WS_CHILD | WS_VISIBLE | WS_VSCROLL | ES_MULTILINE | ES_AUTOVSCROLL | ES_WANTRETURN;
    if (!isWordWrap) editStyle |= ES_AUTOHSCROLL | WS_HSCROLL;
```

```
    HWND hOld = hEdit;
    hEdit = CreateWindowEx(WS_EX_CLIENTEDGE, "EDIT", buffer ? buffer : "", editStyle,
                           0, 0, 0, 0, hwndParent, NULL, GetModuleHandle(NULL), NULL);

    SendMessage(hEdit, WM_SETFONT, (WPARAM)hCurrentFont, TRUE);
    oldEditProc = (WNDPROC)SetWindowLongPtr(hEdit, GWLP_WNDPROC, (LONG_PTR)EditSubProc);

    DestroyWindow(hOld);

    RECT rc; GetClientRect(hwndParent, &rc);
    MoveWindow(hEdit, 0, 0, rc.right, rc.bottom, TRUE);
    if (buffer) free(buffer);
}

// ============================================================================
// --- ABOUT DIALOG ---
// ============================================================================

LRESULT CALLBACK AboutWndProc(HWND hDlg, UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            static HFONT hTitleFont = NULL, hBodyFont = NULL;
            if (!hBodyFont)
                hBodyFont = CreateFont(-13, 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE,
                    ANSI_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
                    DEFAULT_QUALITY, DEFAULT_PITCH | FF_SWISS, "Segoe UI");
            if (!hTitleFont)
                hTitleFont = CreateFont(-19, 0, 0, 0, FW_BOLD, FALSE, FALSE, FALSE,
                    ANSI_CHARSET, OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS,
                    DEFAULT_QUALITY, DEFAULT_PITCH | FF_SWISS, "Segoe UI");

            HWND hTitle = CreateWindow("STATIC",
                "Digispark Keyboard Scripter IDE",
                WS_CHILD | WS_VISIBLE | SS_LEFT | SS_NOPREFIX,
                22, 18, 406, 26, hDlg, NULL, GetModuleHandle(NULL), NULL);
            SendMessage(hTitle, WM_SETFONT, (WPARAM)hTitleFont, TRUE);

            HWND hText = CreateWindow("STATIC",
                "Version " APP_VERSION "\n\n"
                "Creator & Project Lead:\n"
                "    Mike Young\n\n"
                "Software Engineering:\n"
                "    Google Gemini and Anthropic Claude\n\n"
                "GitHub repository:\n"
                "    " GITHUB_URL,
                WS_CHILD | WS_VISIBLE | SS_LEFT | SS_NOPREFIX,
                22, 50, 406, 188, hDlg, NULL, GetModuleHandle(NULL), NULL);
            SendMessage(hText, WM_SETFONT, (WPARAM)hBodyFont, TRUE);

            HWND hVisit = CreateWindow("BUTTON", "Visit GitHub Link",
                WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
                22, 255, 150, 30, hDlg, (HMENU)(UINT_PTR)IDC_ABOUT_VISIT,
                GetModuleHandle(NULL), NULL);
            SendMessage(hVisit, WM_SETFONT, (WPARAM)hBodyFont, TRUE);

            HWND hClose = CreateWindow("BUTTON", "Close",
                WS_CHILD | WS_VISIBLE | BS_PUSHBUTTON,
```

```
                333, 255, 95, 30, hDlg, (HMENU)(UINT_PTR)IDC_ABOUT_CLOSE,
                GetModuleHandle(NULL), NULL);
            SendMessage(hClose, WM_SETFONT, (WPARAM)hBodyFont, TRUE);
            break;
        }

        case WM_COMMAND:
            switch (LOWORD(wParam)) {
                case IDC_ABOUT_VISIT:
                    ShellExecute(hDlg, "open", GITHUB_URL, NULL, NULL, SW_SHOWNORMAL);
                    break;
                case IDC_ABOUT_CLOSE:
                    DestroyWindow(hDlg);
                    break;
            }
            break;

        case WM_CLOSE:
            DestroyWindow(hDlg);
            break;

        default:
            return DefWindowProc(hDlg, msg, wParam, lParam);
    }
    return 0;
}

void ShowAboutDialog(HWND hParent) {
    int w = 450, h = 340;
    RECT rp;
    GetWindowRect(hParent, &rp);
    int x = rp.left + ((rp.right - rp.left) - w) / 2;
    int y = rp.top  + ((rp.bottom - rp.top) - h) / 2;
    if (x < 0) x = 0;
    if (y < 0) y = 0;

    CreateWindowEx(WS_EX_DLGMODALFRAME, "AboutDlgClass",
        "About Digispark Keyboard Scripter IDE",
        WS_POPUP | WS_CAPTION | WS_SYSMENU | WS_VISIBLE,
        x, y, w, h, hParent, NULL, GetModuleHandle(NULL), NULL);
}

// Maps a safePrint delay value to its Typing-Delay menu command ID (-1 if none).
int DelayToMenuId(int d) {
    switch (d) {
        case 0:  return IDM_DELAY_0;
        case 1:  return IDM_DELAY_1;
        case 5:  return IDM_DELAY_5;
        case 10: return IDM_DELAY_10;
        case 15: return IDM_DELAY_15;
        default: return -1;
    }
}

// ============================================================================
// --- WINDOWS GUI EVENT LOOP ---
// ============================================================================
```

```
LRESULT CALLBACK WndProc(HWND hwnd, UINT msg, WPARAM wParam, LPARAM lParam) {
    switch (msg) {
        case WM_CREATE: {
            HMENU hMenu = CreateMenu();
            HMENU hFileMenu = CreatePopupMenu();
            HMENU hBuildMenu = CreatePopupMenu();
            HMENU hOptionsMenu = CreatePopupMenu();

            AppendMenu(hFileMenu, MF_STRING, IDM_NEW, "New");
            AppendMenu(hFileMenu, MF_STRING, IDM_OPEN, "Open...");
            AppendMenu(hFileMenu, MF_STRING, IDM_SAVE, "Save");
            AppendMenu(hFileMenu, MF_STRING, IDM_SAVE_AS, "Save As...");
            AppendMenu(hMenu, MF_POPUP, (UINT_PTR)hFileMenu, "File");

            AppendMenu(hBuildMenu, MF_STRING, IDM_FLASH, "Compile && Flash Script");
            AppendMenu(hMenu, MF_POPUP, (UINT_PTR)hBuildMenu, "Build");

            AppendMenu(hOptionsMenu, MF_STRING | (isDarkMode ? MF_CHECKED : 0), IDM_TOGGLE_DARK, "Dark
Mode");
            AppendMenu(hOptionsMenu, MF_STRING | (isWordWrap ? MF_CHECKED : 0), IDM_TOGGLE_WRAP, "Word
Wrap");

            // Typing Delay submenu (safePrint inter-key delay presets)
            gDelayMenu = CreatePopupMenu();
            AppendMenu(gDelayMenu, MF_STRING, IDM_DELAY_0,  "No Delay");
            AppendMenu(gDelayMenu, MF_STRING, IDM_DELAY_1,  "1 ms");
            AppendMenu(gDelayMenu, MF_STRING, IDM_DELAY_5,  "5 ms");
            AppendMenu(gDelayMenu, MF_STRING, IDM_DELAY_10, "10 ms");
            AppendMenu(gDelayMenu, MF_STRING, IDM_DELAY_15, "15 ms");
            AppendMenu(hOptionsMenu, MF_POPUP, (UINT_PTR)gDelayMenu, "Typing Delay");
            {
                int sel = DelayToMenuId(safePrintDelay);
                if (sel != -1)
                    CheckMenuRadioItem(gDelayMenu, IDM_DELAY_0, IDM_DELAY_15, (UINT)sel, MF_BYCOMMAND);
            }

            AppendMenu(hOptionsMenu, MF_SEPARATOR, 0, NULL);
            AppendMenu(hOptionsMenu, MF_STRING, IDM_ABOUT, "About...");
            AppendMenu(hMenu, MF_POPUP, (UINT_PTR)hOptionsMenu, "Options");

            SetMenu(hwnd, hMenu);

            DWORD editStyle = WS_CHILD | WS_VISIBLE | WS_VSCROLL | ES_MULTILINE | ES_AUTOVSCROLL |
ES_WANTRETURN;
            if (!isWordWrap) editStyle |= ES_AUTOHSCROLL | WS_HSCROLL;

            hEdit = CreateWindowEx(WS_EX_CLIENTEDGE, "EDIT", "",
                editStyle,
                0, 0, 0, 0, hwnd, NULL, GetModuleHandle(NULL), NULL);

            // Subclass the edit control to handle custom pasting logic
            oldEditProc = (WNDPROC)SetWindowLongPtr(hEdit, GWLP_WNDPROC, (LONG_PTR)EditSubProc);

            hEditBkBrush = CreateSolidBrush(RGB(30, 30, 30));

            HFONT hFont = CreateFont(18, 0, 0, 0, FW_NORMAL, FALSE, FALSE, FALSE, ANSI_CHARSET,
                                     OUT_DEFAULT_PRECIS, CLIP_DEFAULT_PRECIS, DEFAULT_QUALITY,
                                     DEFAULT_PITCH | FF_SWISS, "Consolas");
```

```
            SendMessage(hEdit, WM_SETFONT, (WPARAM)hFont, TRUE);
            break;
        }

        case WM_CTLCOLOREDIT: {
            HDC hdc = (HDC)wParam;
            if (isDarkMode) {
                SetTextColor(hdc, RGB(220, 220, 220));
                SetBkColor(hdc, RGB(30, 30, 30));
                return (INT_PTR)hEditBkBrush;
            } else {
                SetTextColor(hdc, RGB(0, 0, 0));
                SetBkColor(hdc, RGB(255, 255, 255));
                return (INT_PTR)GetStockObject(WHITE_BRUSH);
            }
        }

        case WM_SIZE: {
            MoveWindow(hEdit, 0, 0, LOWORD(lParam), HIWORD(lParam), TRUE);
            break;
        }

        case WM_COMMAND: {
            switch (LOWORD(wParam)) {

                case IDM_NEW:
                    SetWindowText(hEdit, "");
                    strcpy(currentFile, "");
                    SetWindowText(hwnd, "Digispark Keyboard Scripter IDE - Untitled");
                    break;

                case IDM_OPEN: {
                    OPENFILENAME ofn;
                    ZeroMemory(&ofn, sizeof(ofn));
                    ofn.lStructSize = sizeof(ofn);
                    ofn.hwndOwner = hwnd;
                    ofn.lpstrFilter = "Text Files (*.txt)\0*.txt\0All Files (*.*)\0*.*\0";
                    ofn.lpstrFile = currentFile;
                    ofn.nMaxFile = MAX_PATH;
                    ofn.Flags = OFN_EXPLORER | OFN_FILEMUSTEXIST;

                    if (GetOpenFileName(&ofn)) {
                        LoadFile(hwnd, currentFile);
                        char title[MAX_PATH + 64];
                        snprintf(title, sizeof(title), "Digispark Keyboard Scripter IDE - %s",
currentFile);
                        SetWindowText(hwnd, title);
                    }
                    break;
                }

                case IDM_SAVE: {
                    if (strlen(currentFile) > 0) {
                        SaveFile(hwnd, currentFile);
                    } else {
                        SendMessage(hwnd, WM_COMMAND, IDM_SAVE_AS, 0);
                    }
                    break;
```

```
                }

                case IDM_SAVE_AS: {
                    OPENFILENAME ofn;
                    ZeroMemory(&ofn, sizeof(ofn));
                    ofn.lStructSize = sizeof(ofn);
                    ofn.hwndOwner = hwnd;
                    ofn.lpstrFilter = "Text Files (*.txt)\0*.txt\0All Files (*.*)\0*.*\0";
                    ofn.lpstrFile = currentFile;
                    ofn.nMaxFile = MAX_PATH;
                    ofn.Flags = OFN_EXPLORER | OFN_PATHMUSTEXIST | OFN_HIDEREADONLY | OFN_OVERWRITEPROMPT;
                    ofn.lpstrDefExt = "txt";

                    if (GetSaveFileName(&ofn)) {
                        SaveFile(hwnd, currentFile);
                        char title[MAX_PATH + 64];
                        snprintf(title, sizeof(title), "Digispark Keyboard Scripter IDE - %s",
currentFile);
                        SetWindowText(hwnd, title);
                    }
                    break;
                }

                case IDM_TOGGLE_DARK:
                    isDarkMode = !isDarkMode;
                    CheckMenuItem(GetMenu(hwnd), IDM_TOGGLE_DARK, isDarkMode ? MF_CHECKED : MF_UNCHECKED);
                    InvalidateRect(hEdit, NULL, TRUE);
                    break;

                case IDM_TOGGLE_WRAP:
                    isWordWrap = !isWordWrap;
                    CheckMenuItem(GetMenu(hwnd), IDM_TOGGLE_WRAP, isWordWrap ? MF_CHECKED : MF_UNCHECKED);
                    RecreateEditor(hwnd);
                    break;

                case IDM_ABOUT:
                    ShowAboutDialog(hwnd);
                    break;

                case IDM_DELAY_0:
                case IDM_DELAY_1:
                case IDM_DELAY_5:
                case IDM_DELAY_10:
                case IDM_DELAY_15: {
                    UINT id = LOWORD(wParam);
                    switch (id) {
                        case IDM_DELAY_0:  safePrintDelay = 0;  break;
                        case IDM_DELAY_1:  safePrintDelay = 1;  break;
                        case IDM_DELAY_5:  safePrintDelay = 5;  break;
                        case IDM_DELAY_10: safePrintDelay = 10; break;
                        case IDM_DELAY_15: safePrintDelay = 15; break;
                    }
                    CheckMenuRadioItem(gDelayMenu, IDM_DELAY_0, IDM_DELAY_15, id, MF_BYCOMMAND);
                    break;
                }

                case IDM_FLASH: {
                    SaveFile(hwnd, "temp_payload.txt");
```

```
                    if (transpile("temp_payload.txt")) {
                        char buildOut[8192];
                        int compileStatus = RunCommandCapture(
                            "arduino-cli compile --config-file arduino-cli.yaml --fqbn
digistump:avr:digispark-tiny sketch",
                            buildOut, sizeof(buildOut));

                        if (compileStatus != 0) {
                            int tooBig = (strstr(buildOut, "overflowed") != NULL)
                                      || (strstr(buildOut, "will not fit in region") != NULL)
                                      || (strstr(buildOut, "not within region") != NULL);

                            char errMsg[8192 + 512];
                            if (tooBig) {
                                snprintf(errMsg, sizeof(errMsg),
                                    "Compilation failed: the sketch is TOO LARGE for the Digispark's
flash.\n\n"
                                    "Shorten the script, or move long payloads to a downloaded stage.\n\n"
                                    "--- Compiler output ---\n%s", buildOut);
                            } else {
                                snprintf(errMsg, sizeof(errMsg),
                                    "Compilation failed!\n\nThere is likely a syntax error in your
script.\n\n"
                                    "--- Compiler output ---\n%s", buildOut);
                            }
                            MessageBox(hwnd, errMsg, "Build Error", MB_ICONERROR);
                            break;
                        }

                        char sketchLine[256] = "", ramLine[256] = "";
                        extract_line(buildOut, "Sketch uses", sketchLine, sizeof(sketchLine));
                        extract_line(buildOut, "Global variables", ramLine, sizeof(ramLine));

                        // Enforce the practical flash ceiling BEFORE uploading.
                        // arduino-cli may report a higher "Maximum" than the
                        // bootloader actually leaves usable, so a sketch that
                        // "fits" per the compiler can still fail on hardware.
                        int usedBytes = 0;
                        if (sscanf(sketchLine, "Sketch uses %d", &usedBytes) == 1
                                && usedBytes > SAFE_FLASH_MAX) {
                            char warn[640];
                            snprintf(warn, sizeof(warn),
                                "Sketch is %d bytes, over the reliable limit of %d bytes for this "
                                "Digispark's bootloader.\n\n%s\n\n"
                                "The compiler reports it fits, but payloads above %d bytes often "
                                "fail to upload or run correctly, so the upload was cancelled.\n\n"
                                "Please shorten the script (or move long payloads to a downloaded stage).",
                                usedBytes, SAFE_FLASH_MAX, sketchLine, SAFE_FLASH_MAX);
                            MessageBox(hwnd, warn, "Sketch Too Large (Safety Limit)", MB_OK |
MB_ICONERROR);
                            break;
                        }

                        char okMsg[1024];
                        snprintf(okMsg, sizeof(okMsg),
                            "Compilation successful!\n\n%s\n%s\n\n"
```

```
                            "Unplug your Digispark, click OK, and then plug it back in within 60 seconds to
start flashing.",
                            sketchLine[0] ? sketchLine : "(flash usage unavailable)",
                            ramLine[0]    ? ramLine    : "");
                        MessageBox(hwnd, okMsg, "Ready to Flash", MB_OK | MB_ICONINFORMATION);

                        int uploadStatus = RunCommandHidden("arduino-cli upload --config-file
arduino-cli.yaml --fqbn digistump:avr:digispark-tiny sketch");

                        if (uploadStatus != 0) {
                            MessageBox(hwnd, "Flashing failed! Did the Digispark time out?", "Flashing
Failed", MB_ICONERROR);
                        } else {
                            MessageBox(hwnd, "Script flashed successfully!", "Flashing Succeeded", MB_OK);
                        }

                    } else {
                        MessageBox(hwnd, "Transpilation failed. Check your script for invalid commands.",
"Transpilation Failed", MB_ICONERROR);
                    }
                    break;
                }
            }
            break;
        }

        case WM_DESTROY:
            SaveSettings();
            if (hEditBkBrush) DeleteObject(hEditBkBrush);
            PostQuitMessage(0);
            break;

        default:
            return DefWindowProc(hwnd, msg, wParam, lParam);
    }
    return 0;
}

// ============================================================================
// --- APPLICATION ENTRY POINT ---
// ============================================================================
int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {
    InitIniPath();
    LoadSettings();

    WNDCLASS wc = {0};
    wc.lpfnWndProc = WndProc;
    wc.hInstance = hInstance;
    wc.lpszClassName = "DigiIDEClass";
    wc.hCursor = LoadCursor(NULL, IDC_ARROW);
    wc.hbrBackground = (HBRUSH)(COLOR_WINDOW + 1);

    RegisterClass(&wc);

    WNDCLASS wcAbout = {0};
    wcAbout.lpfnWndProc   = AboutWndProc;
    wcAbout.hInstance     = hInstance;
    wcAbout.lpszClassName = "AboutDlgClass";
```

```
    wcAbout.hCursor       = LoadCursor(NULL, IDC_ARROW);
    wcAbout.hbrBackground = (HBRUSH)(COLOR_BTNFACE + 1);
    RegisterClass(&wcAbout);

    HWND hwnd = CreateWindowEx(0, "DigiIDEClass", "Digispark Keyboard Scripter IDE - Untitled",
        WS_OVERLAPPEDWINDOW, CW_USEDEFAULT, CW_USEDEFAULT, 800, 600,
        NULL, NULL, hInstance, NULL);

    ShowWindow(hwnd, nCmdShow);

    MSG msg;
    while (GetMessage(&msg, NULL, 0, 0)) {
        TranslateMessage(&msg);
        DispatchMessage(&msg);
    }
    return msg.wParam;
}
```

Save that file. Note that this is the LICENSE file…

LICENSE

```
MIT License

Copyright (c) 2026 Mike Young

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

Go to the command prompt, make sure you are in the “tcc” folder, then run this command to compile that C source code file into a Windows GUI application…

cmd command

```
tcc dks.c -o dks.exe -luser32 -lgdi32 -lcomdlg32 -Wl,-subsystem=windows
```

That command should succeed and generate the file “dks.exe” in the “tcc” folder. Run this command to move the “dks.exe” file to the parent folder, so it can run alongside “arduino-cli.exe”…

cmd command

```
move dks.exe ..
```

Next, launch “dks.exe” from the “dks” folder, then copy and paste the following sample script into the editor and save it as “payload.txt”…

payload.txt

```
defaultdelay 300   # delay 300ms between commands

waitblink(1)   # blink the red LED once every 2s and wait for button press
keys(gui, r)        # press [Windows] + [R]
string("notepad")   # type "notepad"
key(enter)          # press [Enter]
delay(1000)               # wait 1 second for Notepad to appear
string("Hello, world!")   # type "Hello, world!"
led(on)                   # turn on red LED to indicate script is done running
```

Click on **Build** → **Compile & Flash Script**. The script will be transpiled into an ino file. You should get a notification dialog window stating **Compilation successful!** Unplug the Digispark, click OK to the notification, then re-insert the Digispark. The system should proceed with flashing the compiled ino to the Digispark. When it finishes you should see another notification dialog window appear stating **Script flashed successfully!** Click **OK**. The payload should then run.

Note that error checking of the script is very basic. A simple syntax error in your keyboard script might still compile and result in unpredictable or omitted output. In some cases, severe syntax errors may cause the generated ino file to not compile, resulting in an error notification. Note too that you can debug your script by opening and analyzing the generated ino file “sketch.ino”, located in the “sketch” folder.

Semi-Portable Use

The generated application isn’t completely portable, as the Digistump drivers need to be installed on any Windows computer that will be used for uploading compiled scripts. Additionally, regarding the “mdns-discovery.exe” issue, while that’s just a one-time issue, as firewall rules get saved when you make a choice when prompted about that executable, that problem can be suppressed from the beginning by replacing “mdns-discovery.exe” with a dummy version of the program that does nothing.

Start by creating a dummy program called “dummy.c” with the following content, saved to the tcc directory…

dummy.c

```
#include <windows.h>

int WINAPI WinMain(HINSTANCE hInstance, HINSTANCE hPrevInstance, LPSTR lpCmdLine, int nCmdShow) {
    return 0;   // do absolutely nothing and exit successfully
}
```

In the command prompt, change directory to the tcc directory and compile that program with this command…

cmd command

```
tcc dummy.c -o mdns-discovery.exe
```

Move the generated dummy “mdns-discovery.exe” file to…

…\dks\portable_data\packages\builtin\tools\mdns-discovery\1.0.12

When prompted to replace it, proceed with doing so.

Create a batch file named “Install Digistump Drivers.cmd” that resides in the “dks” folder, with the following content…

"Install Digistump Drivers.cmd"

```
@echo off
"%~dp0portable_data\packages\digistump\tools\micronucleus\2.0a4\install.exe"
```

The contents of the “dks” folder can now be compressed into a zip file, copied to another system, extracted, and all that needs to be performed is to install the Digistump drivers. Installing the Digistump drivers can be performed by running the installer, which is located here…

…\dks\portable_data\packages\digistump\tools\micronucleus\2.0a4\install.exe

Upon doing that, you should be able to run “dks.exe” and proceed with writing, compiling, and uploading scripts to the Digispark. Though instead of navigating to that folder, just run the “Install Digistump Drivers.cmd” batch file we just created, from the “dks” folder.

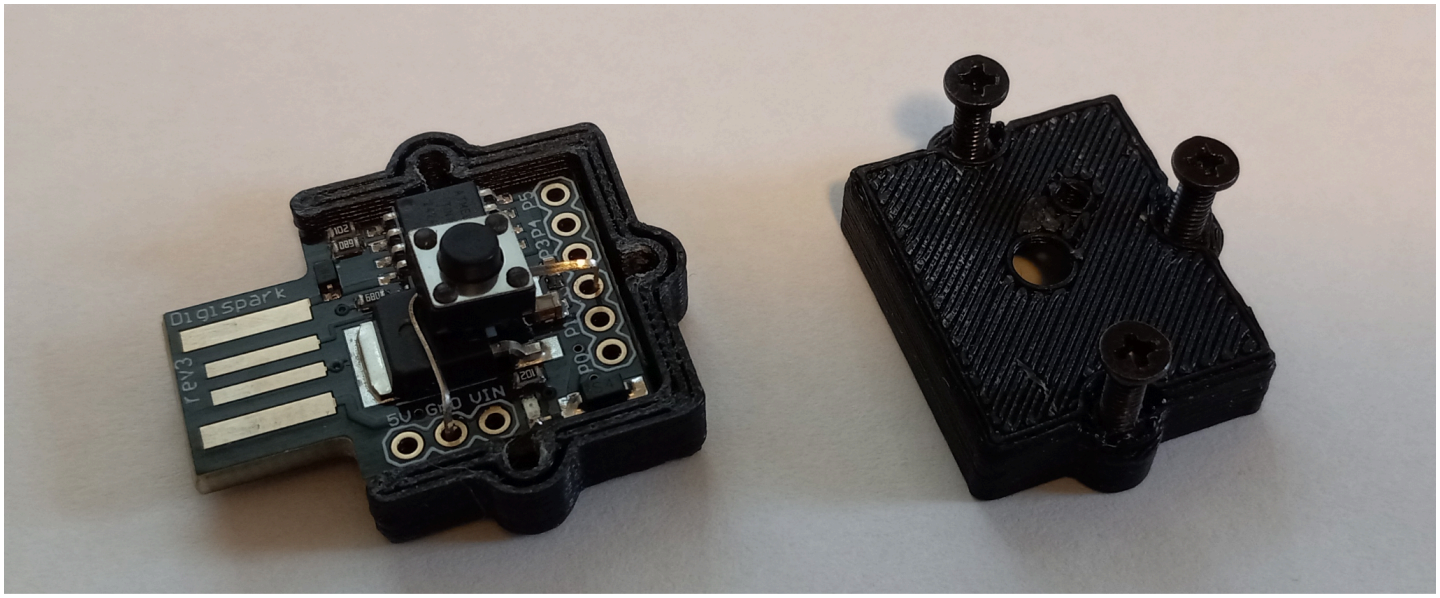
Note that the command to forget about Windows Firewall settings for “mdns-discovery.exe” is (replace **PATH** according to your specific case)…

cmd command

```
netsh advfirewall firewall delete rule name=all
program="PATH\dks\portable_data\packages\builtin\tools\mdns-discovery\1.0.12\mdns-discovery.exe"
```

Additional Hardware Setup

Print the case pieces. Connect a 6x6x4mm tactile pushbutton switch (through-hole, SPST-NO) with long legs to GND and P2 on the Digispark. In my case, I just bend the pins into the GPIO holes and that seems to work reliably due to the friction fit and compression of the case, but applying some solder may be advisable. Position the switch center of the board so that it will fit into the cavity of the top portion of the case. Fit the case pieces together, compress them with your fingers, and screw-in the three M2.5x8mm case screws.



Binary Release Version

The binary release version of the Digispark Keyboard Scripter IDE can be downloaded from this link…

https://www.mikesshorts.com/misc/dks/dks.exe

Once downloaded, run the self extracting executable. It will extract to a folder called “dks”. Open the folder and run “install.cmd”. That will install the Digistump AVR core (ArminJo fork) to the subfolder “portable_data”. It will then run the Digistump driver installation wizard. Proceed with installing the drivers. When complete, you can run “dks.exe”

The Arduino CLI contained in the binary version was obtained from...

https://docs.arduino.cc/arduino-cli/installation/

The ArminJo fork of the Digistump AVR core and Digistump drivers were obtained from:

https://github.com/ArminJo/DigistumpArduino

Specifications for Keyboard Scripting

This language automates keystrokes and hardware states on a Digispark ATtiny85. It is line-based, **case-insensitive** for command names and key identifiers, supports inline comments, and allows up to three simultaneous key presses.

1. Core Commands

Command	Description	Example
`string("text")`	Types the literal string provided inside the quotes. **(Case-sensitive inside quotes)**	`string("notepad.exe")`

`keys(args...)`	The universal key command. Supports 1, 2, or 3 arguments (modifiers first, key last).	`keys(control, alt, delete)`
`key(key)`	An alias for `keys(key)`. Presses and releases a single key.	`key(enter)`
`keydown(key/mod)`	Presses and holds a key or modifier. Modifiers "stack" instantly.	`keydown(control)`
`keyup()`	Releases all currently held keys and modifiers instantly.	`keyup()`
`delay(ms)`	Pauses execution for the specified number of milliseconds.	`delay(1000)`
`defaultdelay ms`	Sets a global delay (in ms) added after every keyboard action. Increase this value slightly if keypresses are being missed or mistyped.	`defaultdelay 200`
`led(on)`	Turns on the Digispark's built-in status LED (Pin 1).	`led(on)`
`led(off)`	Turns off the Digispark's built-in status LED.	`led(off)`
`wait()`	Pauses the script infinitely until the physical button (wired to Pin 2 and GND) is pressed.	`wait()`
`waitblink(x)`	Pauses the script infinitely until the physical button (wired to Pin 2 and GND) is pressed, while blinking x times, pausing 2 seconds, and repeating.	`waitblink(3)`

2. Modifier Keys (MOD)

These are used within `keys()` and `keydown()` functions. They must be placed before the primary key in a multi-key command.

- **GUI** (Windows / Command Key)
- **CONTROL**
- **SHIFT**
- **ALT**

3. Key Definitions (KEY_NAME)

Common identifiers include:

- **Alpha-numeric:** A through Z, 0 through 9
- **Navigation:** UP, DOWN, LEFT, RIGHT, PAGE_UP, PAGE_DOWN, HOME, END
- **Editing:** ENTER, BACKSPACE, DELETE, TAB, SPACE, ESC (or ESCAPE)
- **Function Keys:** F1 through F12
- **System:** PRINTSCREEN, SCROLL_LOCK, PAUSE

4. Scripting Logic & Rules

- **Case Insensitivity:** Commands and key identifiers can be written in uppercase, lowercase, or mixed case (e.g., `DELAY(100)`, `delay(100)`, and `Delay(100)` are identical). **Exception:** Text written inside the quotes of a `string("...")` command will be typed exactly as formatted.
- **Comments:** Use the `#` symbol to add comments. The transpiler will ignore anything on a line after the `#` symbol.
- **Argument Order:** When using multiple keys in `keys()`, the modifiers must come first and the primary key must be last.
    - ✅ **Valid:** `keys(CONTROL, ALT, DELETE)`
    - ❌ **Invalid:** `keys(DELETE, CONTROL, ALT)`
- **Initialization:** Every script starts with an automatic 3-second delay to allow the OS to mount the USB drivers.
- **Stateful Modifiers:** If you use `keydown(SHIFT)`, the Shift key remains held. All subsequent `string()` or `keys()` commands will be typed as if Shift is held until you call `keyup()`.
- **Hardware Hold:** The 3-key version of `keys()` uses a 100ms hardware-level hold to ensure the command is registered by system-level security screens (like the Windows Lock screen).

---

5. Example Payloads

Example 1: Open Notepad and Type

```
defaultdelay 250

keys(gui, r)        # open Run dialog
string("notepad")
key(enter)
delay(1000)   # wait for Notepad to open
string("This payload was deployed via Digispark!")
key(enter)
```

Example 2: Selection & Formatting (Stateful Keys) This script types text, then uses `keydown` to hold modifiers while navigating, mimicking a user highlighting and copying text.

```
defaultdelay 150

string("Automating with Digispark")
keys(enter)

# highlight the line above
keydown(shift)
keys(up)
keyup()
```

```
# copy and paste
keys(control, C)
keys(right)
keys(enter)
keys(control, v)

# system command
keys(control, alt, delete)
```

Example 3: Hardware Interaction (Wait for User) This script waits for the user to push a physical button before executing the payload, using the LED to indicate when it is ready.

```
led(on)   # turn on the LED to signal the USB is mounted and ready
wait()     # pause script execution until the pin 2 button is pressed
led(off)   # turn off the LED so the user knows the attack has started

keys(gui, r)
string("cmd")
keys(enter)
```

Example 4: Bypass NRO in Windows 11 This script opens a command prompt from the Windows 11 OOBE phase and executes command `OOBE\BYPASSNRO`, for bypassing the otherwise mandatory Internet connection and Microsoft Account requirement, to allow setting-up Windows 11 with a local user account. Note the escaped backslash in the string function call. Double quotes and backslashes need to be escaped using a backslash.

```
defaultdelay 300

delay(3000)
keys(shift, f10)
delay(1000)
keydown(alt)
key(tab)
key(tab)
keyup()
string("OOBE\\BYPASSNRO")
key(enter)
led(on)
```

Example 5: Disable OneDrive This script opens a command prompt, kills the OneDrive task, then removes the run entry from the registry for the current user, effectively disabling OneDrive for the currently logged-in user. Note the escaped backslashes and double quotes in the string function call. Double quotes and

backslashes need to be escaped using a backslash. Note the use of cls and echo to communicate the intent of the script, and waitblink(1) for confirmation to proceed by pressing the button on the Digispark.

```
defaultdelay 300

delay(1000)
keys(gui, r)
string("cmd")
key(enter)
delay(1000)
string("cls && echo Press button to proceed with disabling OneDrive")
key(enter)
waitblink(1)
key(escape)
string("taskkill /f /im OneDrive.exe")
key(enter)
delay(1000)
string("reg delete \"HKCU\\Software\\Microsoft\\Windows\\CurrentVersion\\Run\" /v
OneDrive")
key(enter)
delay(500)
key(y)
key(enter)
led(on)
```

Important Limitations

- **Keyboard Layouts:** The device uses a US English HID mapping. If the target computer uses a different layout (e.g., AZERTY or QWERTZ), certain symbols like `/`, `\`, and `@` may not map correctly.
- **Execution:** The script runs exactly once per plug-in. To repeat, unplug and re-insert the device.